

Recursivitatea - Subprograme recursive

Thursday, July 2, 2026 4:06 PM

Recursivitatea = procesul care, prin executie, genereaza aparitia unor procese identice, aflate in legatura directa cu procesul care le genereaza

Subprogramele recursive pot fi atat de tip void, cat si sa returneze o valoare si se caracterizeaza prin autoapel (apelarea functiei in interiorul acesteia)

Elementele necesare intr-un program recursiv:

- Formula de recurenta
- Valoarea initiala cunoscuta (conditia de oprire)

Ex: factorialul unui numar

Analogie matematica: Avem a_n sirul in care pe pozitia n avem factorialul numarului n

Formula de recurenta: $a_n = n * a_{n-1}$ (modul in care termenul curent depinde de termenul/termenii anteriori)

Valoarea initiala: $0! = 1$

In informatica:

-conditia de oprire: $\text{if}(n==0) \rightarrow 0$

-pas recursiv: $n * f(n-1)$

! Nu avem bucle (for, while, do-while) pt programe simple. Practic trebuie sa gandim un algoritm iterativ intr-unul recursiv care urmeaza reguli de recurenta.

Reguli in constructia algoritmului:

1. Conditia de oprire sa fie pusa
2. Consistenta - orice modificare duce la sfarsitul repetitiei
3. Prezenta elementelor mentionate anterior

! Nu intotdeauna recursivitatea este o varianta optima.

Sirul lui Fibonacci:

Formula recursiva: $a_n = a_{n-1} + a_{n-2}$

$f(4)$

$f(3) + f(2)$

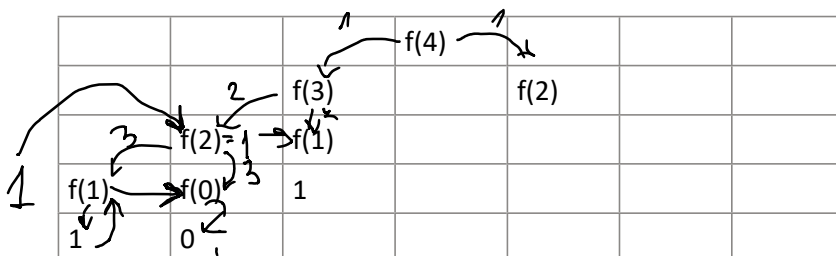
$f(2) + f(1) + f(1) + f(0)$

$f(1) + f(0) + f(1) + f(1) + f(0)$

Cum functioneaza?

Pentru fiecare apel al functiei se adauga pe stiva o zona de memorie in care se memoreaza variabilele locale si parametrii functiei pentru apelul curent.

Aceasta zona va fi in stiva pana la finalul apelului, dupa va elibera stiva



Daca din apelul curent se face un alt apel, se va adauga apelul nou pe stiva intr-o noua zona, care devine accesibila la **revenirea din apel**.

Exemplu factorialul unui numar:

In mod iterativ:

```
int fact_iterativ(int n)
{
    int p=1;
    for(int i=2;i<=n;i++)
        p=p*i;
    return p;
}
```

In mod recursiv:

```
int fact_rec(int n)
{
    if(n==0) return 1; // return-ul ne asigura ca conditia a fost sau nu indeplinita
    return n*fact_rec(n-1);
}
```

```
int main()
{
    int x;
    x=fact_rec(3);
    cout<<x;
}
```

Pas	Continut stiva	Observatii
Int x	☐ <-x=???	Se aloca spatiu pt x in stiva
x=fact_rec(3)	Zona 1: in care e casuta lui x fara valoare ☐ <-x=???	Se aloca o alta zona pt elemente lui fact_rec(3). Cand se ajunge la alt apel (fact_rec(2)) se blocheaza zona 2 si se creeaza o noua zona de memorie
	Zona 2: fact_rec(3) ☐ <-n=3 3*fact_rec(2)	
	Zona 1: in care e casuta lui x fara valoare ☐ <-x=???	
	Zona 2: fact_rec(3) ☐ <-n=3 3*fact_rec(2)	
	Zona 3: fact_rec(2) ☐ <-n=2 2*fact_rec(1)	

	Zona 1: in care e casuta lui x fara valoare ☐ <-x=??? Zona 2: fact_rec(3) ☐ <-n=3 3*fact_rec(2) Zona 3: fact_rec(2) ☐ <-n=2 2*fact_rec(1) Zona 4: fact_rec(1) ☐ <-n=1 1*fact_rec(0)	
	Zona 1: in care e casuta lui x fara valoare ☐ <-x=??? Zona 2: fact_rec(3) ☐ <-n=3 3*fact_rec(2) Zona 3: fact_rec(2) ☐ <-n=2 2*fact_rec(1) Zona 4: fact_rec(1) ☐ <-n=1 1*fact_rec(0) Zona 5: fact_rec(0) = 1 ☐ <-n=0 Return 1	S-a ajuns la conditia de oprire si ne intoarcem la apelurile anterioare in ordine si eliberam zona de memorie curenta.
	Zona 1: in care e casuta lui x fara valoare ☐ <-x=??? Zona 2: fact_rec(3) ☐ <-n=3 3*fact_rec(2) Zona 3: fact_rec(2) ☐ <-n=2 2*fact_rec(1) Zona 4: fact_rec(1) = 1 ☐ <-n=1 1*fact_rec(0) = 1*1=1	

	Zona 5: fact_rec(0) = 1 □<-n=0 Return 1	
	Zona 1: in care e casuta lui x fara valoare □<-x=??? Zona 2: fact_rec(3) □<-n=3 3*fact_rec(2) Zona 3: fact_rec(2) = 2 □<-n=2 2*fact_rec(1) = 2 * 1 = 2 Zona 4: fact_rec(1) = 1 □<-n=1 1*fact_rec(0) Zona 5: fact_rec(0) = 1 □<-n=0 Return 1	
	Zona 1: in care e casuta lui x fara valoare □<-x=??? Zona 2: fact_rec(3) = 6 □<-n=3 3*fact_rec(2) = 3*2=6 Zona 3: fact_rec(2) = 2 □<-n=2 2*fact_rec(1) Zona 4: fact_rec(1) = 1 □<-n=1 1*fact_rec(0) Zona 5: fact_rec(0) = 1 □<-n=0 Return 1	
	Zona 1: in care e casuta lui x fara valoare □<-x=6 Zona 2: fact_rec(3) = 6 □<-n=3 3*fact_rec(2) Zona 3: fact_rec(2) = 2 □<-n=2 2*fact_rec(1)	

	Zona 4: fact_rec(1)=1 □<n=1 1*fact_rec(0) Zona 5: fact_rec(0)=1 □<n=0 Return 1	
--	--	--

Recursivitatea in functie de tipul subprogramului:

Functie de tip void (executa actiuni fara a da un rezultat mai departe)

```
void fact_rec(int n, int& f)
{
    if(n==0)
        f=1;
    else
    {
        fact_rec(n-1,f);
        f=f*n;
    }
}
int main()
{
    int f;
    fact(3, f);
    cout<<f;
}
```

-> apelul se face cu functia de sine statatoare

-> ca sa dam mai departe un rezultat, vom folosi un parametru in care se va da valoarea calculata (parametrii de iesire --- &)

-> folosim if-else cases

Functii care returneaza un rezultat:

```
int fact_rec(int n)
{
    if(n==0) return 1; // return-ul ne asigura ca conditia a fost sau nu indeplinita
    return n*fact_rec(n-1);
}

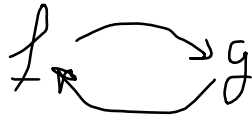
int main()
{
    int x;
    x=fact_rec(3);
    cout<<x;
}
```

Tipuri de recursivitate:

1. Recursivitatea directa: in definitia functiei f se apeleaza functia f



2. Recursivitatea indirecta: in definitia lui f se apeleaza g, iar in definitia lui g se apeleaza f



```

Tip_functie nume_functie(parametrii)
{
    If(Conditie de oprire) instructiune_0//face ceva return sau atribuire
    Else
    {
        Instructiunea_1
        Nume_functie(parametrii)
        Instructiunea_2
    }
}
  
```

Se executa in felul urmator:

Instructiunea 1 se repeta de atatea ori cate apeluri avem

Instructiunea 0

Instructiunea 2 cand se revine din apel, adica de atatea ori de cate ori avem instructiunea 1

4207

$$S=1*2+2*3+ \dots + (n-1)*n = \sum_{k=2}^n k * (k - 1)$$